

Relatório Projeto

Época Normal

Filipe Paredes

202300257

LEI

Computação Paralela e Distribuída

07/2025

Descrição dos algoritmos implementados

Módulo calculo.py

- **is_prime(n):**

Determina se um determinado número, n , é primo ou não. Num momento inicial, o algoritmo testa casos particulares, nomeadamente os números ≥ 1 até 3, e múltiplos de 2 e 3.

É então definido o limite dos divisores até à raiz quadrada de n , e o algoritmo começa a testar os divisores em n , começando por 5, e incrementando de 6 em 6, cobrindo todos os números da forma $6k \pm 1$, que são os únicos candidatos a primos maiores que 3.

- **_worker(start, step, max_prime, lock, stop_time):**

É a função executada por cada processo em `find_max_prime_parallel`. Procura, dentro do tempo definido, pelo maior primo, com um certo 'step', que lhe é atribuído. Posteriormente, compara com o maior primo encontrado pelos outros workers, através de um lock, e se for efetivamente maior, guarda-o na variável.

- **find_max_prime_parallel(timeout):**

Procura pelo maior primo, dentro de um tempo dado, de forma paralela, utilizando processos. Guarda o valor numa variável "Value", gerida por um lock, de modo a poder ser acessada por todos os processos. O número de processos é o número de *cores* disponíveis no processador. Cada processo começa num ímpar diferente e procurará num intervalo diferente de valores, de modo a que mais que um processo procure os mesmos números, sendo que os processos procurarão apenas por números ímpares.

Depois de finalizado o tempo, todos os processos são terminados e é retornado o maior valor encontrado.

- **find_max_prime_sequential(timeout):**

Procura o maior número primo, dentro de um tempo dado, de forma sequencial. Incrementa número a número, e verifica, recorrendo a "is_prime" se cada um desses números é primo, e se aplicável, guarda-o numa variável, que após o término do tempo de procura, é retornado ao utilizador.

- **find_next_twin_primes(n):**

Procura pelos próximos números primos gêmeos mais próximos de n. Itera por todos os números n+1, até encontrar um par de números n+1 e n+3 que sejam ambos primos.

- **is_mersenne_prime(n):**

Verifica se n é um número primo de Mersenne. Exclui automaticamente números inferiores a 2, e calcula o logaritmo quadrado de n+1 como sendo p, verifica então se p e n são primos.

- **prime_factors(n):**

Decompõe n em fatores primos por tentativa de divisão sucessiva, começando em 2 e testando todos os inteiros crescentes até à raiz quadrada de n. Sempre que um fator é encontrado, é adicionado à lista e n é reduzido. No fim, se restar um número maior que 1, é adicionado como o último fator (também primo). Os fatores são devolvidos por ordem crescente.

- **next_prime(n):**

Encontra o próximo número primo maior que n, utilizando a função "is_prime".

- **previous_prime(n):**

Encontra o próximo número primo menor que n, utilizando a função "is_prime". Procura até n ser 0, se não encontrar nenhum, retorna None.

Módulo `criptografia.py`

- **_generate_large_prime(bits):**

Gera um número primo com o número de bits especificado. Usa `getrandbits()` para criar um número aleatório, garante que é ímpar e que o bit mais significativo está definido (para assegurar o tamanho), e testa se é primo. Repete até encontrar um número que seja primo.

- **generate_keys(bits):**

Gera um par de chaves RSA (pública e privada) com segurança baseada no tamanho em bits. Cria dois primos grandes distintos p e q , calcula n como sendo $p \cdot q$ e função ϕ como sendo $(p-1) \cdot (q-1)$. Define o expoente público e ajusta se necessário para garantir que é coprimo com ϕ . Calcula d , o inverso modular do expoente em relação a ϕ , e devolve as chaves públicas.

- **encrypt(message, pub_key):**

Dada uma chave pública, encripta uma mensagem (número inteiro) de tamanho máximo n (primeiro valor da chave pública) utilizando potência modular.

- **decrypt(ciphertext, priv_key):**

Desencripta uma mensagem, dada a sua chave privada, utilizando potência modular.

- **_try_factors():**

Procura um fator de n com tentativa de divisão, começando em $start$ incrementando o $step$. Se encontrar um divisor, guarda os fatores primos e ativa a flag `found_flag`, que permite aos outros processos terminarem.

- **crack_key():**

Tenta descobrir a chave privada RSA a partir da chave pública, decompondo n em paralelo com múltiplos processos. Cada processo explora divisores diferentes. Se a decomposição for bem-sucedida dentro do tempo limite, calcula o inverso modular e valida a chave com um teste de encriptação. Devolve então a chave, se for válida.

Módulo servidor_rpc.py

- **__init__(self, host, port):**

Inicializa o servidor RPC com o endereço e porta indicados.
- **register_functions(self):**

Regista todas as funções públicas (cujo nome não começa por `_`) presentes nos módulos calculo e criptografia. Utiliza a função `register()` para registar cada uma.
- **register(self, name, func):**

Regista uma função no servidor RPC com um nome associado, para poder ser invocada posteriormente.
- **list_functions(self):**

Devolve uma lista detalhada das funções registadas, incluindo o nome, parâmetros e a descrição (retirada da docstring).
- **make_response(self, result, error, id):**

Cria uma resposta válida no formato JSON-RPC 2.0, contendo o id da chamada, o result (caso tenha sucesso) ou o error (em caso de falha).
- **handle_single_request(self, req):**

Processa um pedido individual JSON-RPC 2.0. Valida o pedido, executa a função pedida com os argumentos fornecidos, e retorna a resposta.
- **handle_request(self, request_json):**

Recebe uma string JSON com um ou vários pedidos. Analisa o JSON e processa utilizando `handle_single_request()`, tratando também pedidos em batch e erros de parsing.

- **client_thread(self, conn, addr):**

Lida com uma ligação individual de um cliente. Recebe pedidos, processa-os e envia respostas. Atualiza o número de clientes ativos com um lock para garantir segurança na concorrência.

- **start(self):**

Inicia o servidor RPC, e gere as novas ligações, de modo a atribuir cada uma a uma nova thread. Suporta também o comando shutdown, que interrompe novas ligações e espera que todos os clientes terminem antes de desligar o servidor por completo.

Módulo cliente_rpc

- **__init__(self, host, port):**

Estabelece a ligação a um servidor RPC especificando, utilizando sockets TCP.

- **__getattr__(self, attr):**

Permite invocar funções remotas como se fossem métodos locais, tratando automaticamente argumentos posicionais e nomeados.

- **invoke(self, function, arguments):**

Envia um pedido seguindo o protocolo JSON-RPC 2.0 para o servidor para invocar uma função remota com os argumentos fornecidos. Gera um ID único para cada chamada. Trata a resposta e levanta erros, se existirem.

- **dynamic_menu(menu):**

Permite ao utilizador interagir com o servidor RPC através de um menu de texto. Primeiro, obtém a lista de funções disponíveis via chamada remota. Apresenta então um menu numerado com o nome, parâmetros e descrição de cada função. Após a escolha do utilizador, solicita os valores dos parâmetros, distinguindo entre obrigatórios e opcionais. Os argumentos são organizados e enviados numa nova chamada remota à função escolhida. O resultado é exibido no ecrã. O processo repete-se até o utilizador optar por terminar o programa.

Estratégias de paralelização adotadas

Para a função `find_max_prime_parallel()`, optou-se por utilizar processos, sendo que cada worker começará num ponto diferente e salta por múltiplos de `cpu_count()`, de modo a que diferentes workers não procurem pelo mesmos números.

Para a função `crack_key()`, também se optou por utilizar processos paralelos para decompor `n`, de modo a dividir o espaço de procura pelos diferentes workers. Foi utilizada uma flag partilhada (`found_flag`) que sinaliza se um fator já foi encontrado, permitindo aos restantes processos encerrarem assim que a factorização for bem sucedida.

Já para o servidor RPC, utilizou-se threads para gerir múltiplos clientes em simultâneo. Cada ligação é tratada por um thread dedicado, o que permite um serviço concorrente eficiente. Foi também utilizado um lock, que protege variáveis partilhadas, de modo a garantir consistência sem grandes custos de desempenho.

Comparação entre versões sequenciais e paralelas

Para testar as duas versões da função `find_max_prime`, foram testados 3 diferentes tempos máximos de procura, executando cada teste 5 vezes cada vezes. Abaixo as tabela com os resultados.

Nº Teste	Tempo utilizado	Resultado Sequencial	Resultado Paralelo
1	1s	932257	953321
2	1s	939451	957043
3	1s	924361	923653
4	1s	937253	960383
5	1s	921121	1001123

Nº Teste	Tempo utilizado	Resultado Sequencial	Resultado Paralelo
1	3s	2071159	2920987
2	3s	2061313	2939297
3	3s	2062513	2933083
4	3s	2053943	2923469
5	3s	2068877	2799803

Nº Teste	Tempo utilizado	Resultado Sequencial	Resultado Paralelo
1	5s	2995469	4835731
2	5s	3000299	4937329
3	5s	2981711	4875881
4	5s	2985113	4919023
5	5s	2956451	4685567

Com base nos resultados obtidos, podemos então afirmar a versão paralela foi superior em todos os tempos, mas, quanto maior o tempo de procura, melhor é a versão paralela em relação à sequencial.

Resultados de desempenho

find_max_prime:

Versão	Melhor Resultado	Tempo
find_max_prime_sequential	4917113	10s
find_max_prime_parallel (10 núcleos)	10000373	10s

crack_key:

- Bits da chave: 32
- Melhor tempo: 0.2346s

Verificou-se que o tempo necessário para quebrar uma chave RSA por decomposição depende **exponencialmente** do número de bits de n . À medida que os bits aumentaram, o espaço de procura de divisores cresceu drasticamente, tornando o ataque rapidamente impraticável.

Resultados dos Pylint

Após algumas execuções do Pylint e consequentes correções, as melhores pontuações obtidas foram:

- calculo.py: 10/10
- criptografia.py: 10/10
- servidor_rpc.py: 9.13/10
- cliente_rpc.py: 9.31/10

Totalizando uma média final de todo o projeto de 9,61/10.

Testes fornecidos e testes adicionais

Todos os testes foram feitos utilizando a palavra-chave “assert”.

- **test_is_prime():** Testa se a função “is_prime()” determina corretamente os números primos e não primos com valores conhecidos.
- **test_next_previous_prime():** Testa se as funções “next_prime()” e “previous_prime()” determinam corretamente os próximos números primos, com valores conhecidos.
- **test_find_next_twin_primes():** Testa se a função “find_next_twin_primes()” determina corretamente os próximos primos gêmeos com valores conhecidos.
- **test_is_mersenne_prime():** Testa se a função “is_mersenne_prime()” determina corretamente os números primos de mersenne com valores conhecidos.
- **test_prime_factors():** Testa se a função “prime_factors()” determina corretamente os fatores primos com valores conhecidos.

- **test_find_max_prime_timed():** Testa, com um tempo aleatório de 1 a 5, as funções “find_max_prime_sequential()” e “find_max_prime_parallel()”, apresentando posteriormente o resultado de ambas para comparação de resultados.
- **test_generate_keys_and_encrypt_decrypt():** Testa as funções “generate_keys()”, “encrypt()” e “decrypt()”, utilizando uma chave de 16 bits, encriptando uma mensagem, desencriptando-a, e comparando o resultado com a mensagem originalmente encriptada.
- **test_encrypt_invalid_message():** Testa a função “encrypt()”, passando como argumento uma mensagem inválida, e verificando se é levantado corretamente o erro.
- **test_crack_key():** Testa a função “crack_key()”, encriptando uma mensagem, voltando a descriptar com a chave privada obtida, e verificando se a mensagem descriptada coincide com a mensagem original.
- **test_rpc():** Testa a conexão do cliente ao servidor e execução de métodos remotos como se fossem métodos locais, e testa múltiplas ligações simultâneas ao servidor, utilizando threads.